Rapport de projet : Smart Skin Analyzer

Oct 23, 2026 · @ZEUS

Résumé

Le logiciel du miroir est terminé et tourne sur le Raspberry Pi 4 : il analyse une photo du visage avec deux modèles d'apprentissage profond entraînés pour ce projet, et lit les capteurs d'un Arduino Uno. Du côté matériel, la caméra et le capteur d'humidité fonctionnent ; le thermomètre infrarouge n'est pas encore détecté (fil SDA à reprendre) et le capteur de distance n'est pas encore monté.

Le miroir mesure quatre critères visibles de la peau (acné, rides, taches, pores), les combine en un score sur 100, ajoute la température et l'humidité de l'air, estime un type de peau à partir des capteurs et affiche des conseils non médicaux.

Élément	Méthode	Résultat mesuré sur des données jamais vues
Acné	YOLO11s entraîné sur ACNE04 (1 457 photos, 18 983 lésions)	mAP@50 = 0,34 ; précision 0,44 ; rappel 0,40 ; corrélation 0,75 entre boutons détectés et annotés par photo
Rides	U-Net (encodeur ResNet-50) entraîné sur FFHQ-Wrinkle (1 000 masques manuels)	IoU = 0,47 ; F1 = 0,64
Taches et pores	Vision par ordinateur classique (contraste local)	Défauts injectés retrouvés à 100 % ; seuils encore à calibrer sur la caméra du miroir
Type de peau	Règles du binôme (température de la peau + humidité)	Identiques au code d'origine sur 200 lignes sur 200 ; estimation indicative
Temps d'analyse	Raspberry Pi 4, sans accélérateur	114 s par analyse (environ 5 s sur le PC de développement)

Tout le code, les modèles et ce suivi sont publiés dans le dépôt khaidermohamedd/smart-skin-analyzer.

1. Contexte et objectifs

Le projet est un miroir intelligent qui analyse la peau sans contact : une caméra photographie le visage, des modèles d'IA mesurent l'état de la peau et des capteurs ajoutent des mesures physiques. Il réunit deux travaux de Master : l'analyse d'image (ce rapport) et la partie capteurs réalisée par le binôme.

Le projet a démarré de trois documents fournis au départ :

- *Rapport_IoT_Scanner_Peau_Hydratation_Sebum.pdf* : première idée d'un scanner à capteur capacitif (hydratation) et papier buvard (sébum), avec classification du type de peau par règles.
- *iot_materiel_final.pdf* : la liste de matériel retenue pour le miroir (Raspberry Pi 4, caméra, MLX90614, VL53L0X/VL53L1X, DHT22, écran HDMI, miroir sans tain, LED RGB).
- La demande de départ : un modèle d'analyse du visage pour l'acné, les rides et les taches ou trous, en commençant par le logiciel.

Objectifs retenus :

1. Détecter et mesurer quatre critères visibles : acné, rides, taches pigmentaires, pores (petits trous).
2. Utiliser de vrais modèles d'apprentissage profond entraînés sur de vraies bases annotées, avec une précision mesurée honnêtement.
3. Faire tourner l'ensemble sur le Raspberry Pi 4 du miroir, avec des capteurs sans contact.
4. Afficher uniquement des valeurs réelles, issues des capteurs et des modèles.
5. Documenter chaque choix pour le rapport de Master.

Le système donne une indication cosmétique et non un diagnostic médical.

2. Chronologie du projet

Le projet s'est déroulé du 5 au 23 octobre 2026, en quatre temps : logiciel classique, modèles d'IA, matériel, mise en service sur le Raspberry Pi. Les dates suivent l'ordre du travail, du début à aujourd'hui.

Date	Étape	Résultat
5 oct.	Module d'analyse en vision classique (zones du visage, 4 détecteurs)	Fonctionne de bout en bout ; défauts injectés retrouvés (acné, taches, pores 100 %, rides 64 %)

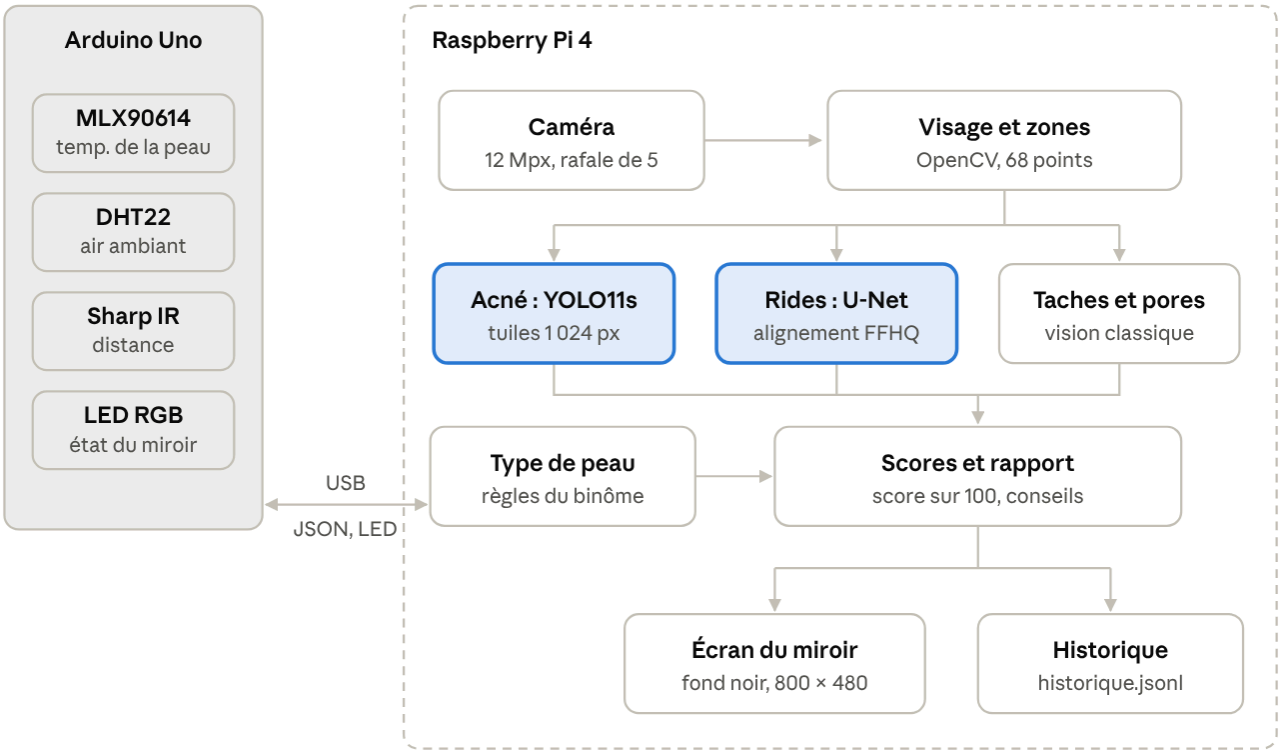
Date	Étape	Résultat
5 oct.	MediaPipe bloqué par Windows (Smart App Control)	Second moteur de points du visage 100 % OpenCV (YuNet + Facemark LBF, 68 points)
5 oct.	Premier guide PDF de réalisation	Guide généré par script, mis à jour ensuite
5 oct.	Choix de vrais modèles et de vraies bases	ACNE04 pour l'acné, FFHQ-Wrinkle pour les rides ; base Roboflow écartée
5 oct.	Disque C: plein, PyTorch bloqué par Windows	Projet déplacé sur D:\skin_ai ; entraînement sous Ubuntu (WSL2) avec la carte graphique
5 oct.	Préparation d'ACNE04 en tuiles de 1 024 px	12 369 tuiles d'entraînement, 42 042 boîtes
5 oct.	Essais : YOLO11m ne tient pas en mémoire, YOLO11s retenu	1 epoch d'essai validée de bout en bout
5 → 6 oct.	Entraînement du modèle d'acné (21:16 → 07:49)	Arrêt automatique à l'epoch 115 ; mAP@50 test = 0,34
6 oct.	Entraînement du modèle de rides (07:51 → 09:42)	IoU test = 0,47 ; F1 = 0,64
11 oct.	Intégration des deux modèles dans l'application, correction du masque de peau, étalonnage des scores	15 boutons détectés au lieu de 0 sur la photo de contrôle ; corrélation 0,75 par photo
11 oct.	Firmware ESP32, programme du miroir, chapitre matériel du guide	Firmware compilé ; miroir testé en simulation
14 → 15 oct.	Dépôt GitHub créé puis rendu public, licences ajoutées	70 fichiers, modèles en Git LFS
15 oct.	Améliorations faites directement sur le Raspberry Pi (rafale photo, clavier, extinction)	Récupérées et fusionnées le 18 oct.
18 oct.	Intégration de la partie capteurs du binôme (Arduino Uno, type de peau)	Règles identiques à l'original sur 200 lignes
18 oct.	Choix : carte Arduino Uno + Sharp IR, température mesurée en 2 temps	Firmware Uno rapide compilé
18 oct.	Mise à jour et test du Raspberry Pi	Analyse complète sur le Pi : 114 s

Date	Étape	Résultat
18 oct.	Fonctionnement sans capteur de distance, diagnostic du câblage	Caméra et DHT22 fonctionnels ; MLX90614 non détecté
23 oct.	Rédaction de ce rapport	

3. Architecture générale

Le Raspberry Pi 4 fait toute l'analyse (caméra, modèles, affichage) ; l'Arduino Uno lit les capteurs et les lui envoie par USB. Les deux modèles d'apprentissage profond (en bleu) traitent l'acné et les rides, la vision classique traite les taches et les pores.

Le Raspberry Pi fait toute l'analyse ; l'Arduino lui envoie les mesures



Architecture du miroir · 2 cartes, 9 blocs logiciels

La photo passe par la détection du visage et le découpage en zones, puis par les trois détecteurs ; leurs mesures et le type de peau calculé à partir des capteurs sont réunis dans un rapport, affiché et enregistré. Le détail de chaque bloc est donné dans les sections 5 à 12.

4. Environnement de développement et contraintes

Le développement s'est fait sur un PC Windows 11 avec carte graphique NVIDIA, mais deux contraintes ont imposé de travailler sous Linux (WSL2) et sur le disque D:.

4.1 Machines

Machine	Caractéristiques	Rôle
PC de développement	Windows 11 Pro, Intel Core i7-10700K (8 cœurs), NVIDIA GeForce RTX 2070 (8 Go)	Code, entraînement des modèles, tests en simulation
Ubuntu 24.04 sous WSL2 (sur le PC)	Python 3.12, PyTorch 2.14 + CUDA 12.6, Ultralytics 8.4, segmentation-models-pytorch 0.5, OpenCV 5.0	Entraînement GPU et exécution du code d'analyse
Raspberry Pi 4 Model B	4 Go de RAM, 4 cœurs, Raspberry Pi OS 13 (Trixie) 64 bits, Python 3.13, onnxruntime 1.31	Miroir : caméra, analyse, affichage

4.2 Contraintes rencontrées

- **Smart App Control** (sécurité de Windows 11, activée) bloque les bibliothèques non signées : MediaPipe dès le premier jour, puis PyTorch, puis scikit-image. Plutôt que de désactiver cette protection, tout le code Python tourne sous Ubuntu (WSL2), installé sur D: ; la carte graphique y reste accessible.
- **Disque C: presque plein** (1,9 Go libres) alors que PyTorch demande environ 5 Go et les bases environ 25 Go : tout le projet est sur `D:\skin_ai`.
- **Dossier dans OneDrive** : les données ont été sorties de OneDrive pour éviter de synchroniser des dizaines de Go vers le cloud.

4.3 Organisation des fichiers

Emplacement	Contenu
`D:\skin_ai\code`	Code source, modèles, firmware, guide (dépôt Git)
`D:\skin_ai\datasets`	ACNE04, FFHQ-Wrinkle, tuiles préparées (non publiés)
`D:\skin_ai\runs`	Journaux, courbes et poids des entraînements

Emplacement	Contenu
`D:\skin_ai\wsl`	Disque virtuel d'Ubuntu (WSL2)
`/home/pi/skin_ai` (Raspberry Pi)	Copie installée du projet

4.4 Suivi de version

Le code est publié dans un dépôt GitHub public (11 commits au 18 octobre). Les six fichiers de modèles (248 Mo au total, dont 130 Mo pour le modèle de rides) sont stockés avec Git LFS, car GitHub refuse les fichiers de plus de 100 Mo. Les bases d'entraînement n'y sont pas : un script les retélécharge. Les secrets (clé Roboflow) sont exclus du dépôt.

5. Phase 1 : module d'analyse en vision classique

La première version détecte les quatre critères par des règles de traitement d'image, sans entraînement. Elle a construit toute la chaîne (visage, zones, scores, affichage) que les modèles d'IA ont ensuite complétée ; ses détecteurs de taches et de pores sont toujours utilisés.

5.1 Points du visage et zones de peau

- **Deux moteurs de points**, choisis automatiquement : MediaPipe Face Landmarker (478 points, le plus précis) et, en secours, OpenCV YuNet (détection du visage) + Facemark LBF (68 points). Sur le Raspberry Pi 4, MediaPipe plante (processeur incompatible) : c'est le moteur OpenCV qui tourne.
- **Normalisation** : la photo est recadrée autour du visage puis mise à l'échelle pour que la distance entre les coins externes des yeux vaille 400 px. Avec une distance réelle moyenne de 90 mm, 1 mm vaut environ 4,4 px : tous les seuils s'expriment en millimètres et ne dépendent plus de la distance à la caméra.
- **Zones** : front, joue gauche, joue droite, nez, menton et pattes d'oie (rectangle orienté au coin externe de chaque œil). Avec 68 points, le front n'a pas de points : il est extrapolé au-dessus des sourcils.
- **Exclusions** : yeux, sourcils et lèvres sont retirés, élargis de quelques pixels.
- **Masque couleur de peau** : les pixels trop éloignés de la couleur médiane des joues (écart Lab > 16) sont retirés, ce qui écarte cheveux et fond. Les trous entièrement entourés de peau sont rebouchés (correctif du 11 octobre, voir section 9).

5.2 Principe des détecteurs

Chaque pixel est comparé à la peau voisine, estimée par un flou gaussien calculé sur les seuls pixels de peau. La détection devient ainsi peu sensible à l'éclairage global et à la carnation.

Critère	Signal utilisé	Règles principales	Mesure
Acné (remplacé par YOLO)	Excès de rouge a* par rapport à la peau voisine	Δa > 7 ; surface 1,5 à 60 mm² ; circularité ⩾ 0,35	Nombre de lésions
Rides (remplacé par U-Net)	Filtre de crêtes de Sato sur la luminance relative (échelles 1,5 ; 2,5 ; 3,5 px)	Réponse > 0,035 ; longueur ⩾ 4 mm ; allongement ⩾ 3	mm de rides par cm²
Taches	Assombrissement L* sans excès de rouge	ΔL > 9 ; Δa < 6 ; surface 2 à 150 mm² ; allongement ⩽ 4	% de surface couverte
Pores / trous	Transformée morphologique black-hat (disque de 9 px)	Contraste > 12 ; de 3 px à 1,2 mm² ; presque ronds ; joues et nez	Pores par cm²

Un pore situé dans un bouton ou une tache n'est pas compté deux fois.

5.3 Scores

Chaque mesure brute v devient un score de sévérité de 0 à 100 par une saturation progressive :

score = 100 × (1 − e^(−v / échelle))

Niveaux : faible sous 25, modéré sous 55, élevé au-delà. Le score global de peau vaut 100 − (0,35 × acné + 0,25 × rides + 0,20 × taches + 0,20 × pores). Les échelles utilisées sont données en section 9.4.

5.4 Contrôle qualité de la photo

L'analyse signale un visage trop petit (moins de 140 px entre les yeux sur la photo d'origine), un éclairage trop faible ou trop fort, et une image floue (variance du laplacien sur la peau inférieure à 40).

5.5 Test par injection

Sans photos annotées pour ces détecteurs, de faux défauts de taille réaliste ont été dessinés sur un vrai visage, puis recherchés.

Défauts dessinés	Retrouvés
6 disques rouges de 3 mm (acné)	6 sur 6 (100 %)
3 disques bruns de 5 mm (taches)	3 sur 3 (100 %)
25 points sombres sur le nez (pores)	25 sur 25 (100 %)
3 lignes sombres sur le front et les pattes d'oie (rides)	64 % des pixels de rides

Ce test prouve que les détecteurs fonctionnent comme prévu, mais pas qu'ils sont précis sur de vraies lésions. C'est ce qui a motivé la phase 2.

6. Choix des données

Deux bases publiques annotées par des humains ont été retenues : ACNE04 pour l'acné et FFHQ-Wrinkle pour les rides. Une troisième base, qui couvrait aussi les taches et les pores, a été examinée puis écartée pour sa qualité.

6.1 Bases étudiées

Base	Contenu	Décision	Raison
ACNE04 (Wu et al., ICCV 2019)	1 457 photos de visages, 18 983 lésions d'acné encadrées, 4 niveaux de sévérité	**Retenue pour l'acné**	Base de référence de la littérature ; photos haute résolution ; annotations de dermatologues
FFHQ-Wrinkle (Moon et al., ICPR 2024)	1 000 masques de rides tracés à la main + 50 000 masques automatiques, sur des visages FFHQ 1 024 × 1 024	**Retenue pour les rides**	Première base publique de segmentation des rides du visage entier ; annotations au pixel près
Skin-Problem-Detection (Roboflow)	2 660 images, 10 classes (acné, taches, pores dilatés, rides, cernes…)	Écartée	Images de 255 × 255 px où les pores sont presque invisibles ; annotations incohérentes (boîtes « cernes » englobant tout le visage, rides en grandes boîtes)

La base Roboflow a été téléchargée et inspectée visuellement avant d'être écartée. Après cette inspection, le choix a été limité à deux modèles profonds (acné et rides) ; les taches et les pores restent en vision classique, faute de base publique fiable.

6.2 ACNE04 : ce qui a été utilisé

- 1 457 photos (dossier Classification) et leurs 1 457 fichiers d'annotations au format VOC (dossier Detection).
- Tailles des photos : 3 112 × 3 456 px (1 107 photos), 4 912 × 3 264 px (119), 5 184 × 3 456 px (19) pour les tailles les plus fréquentes.
- Un bouton n'occupe que 0,5 à 2 % de la largeur d'une photo, d'où la préparation en tuiles décrite en section 7.

6.3 FFHQ-Wrinkle : ce qui a été utilisé

- Les 1 000 masques manuels (identifiants de 00001 à 21035) et les 1 000 photos FFHQ correspondantes.
- 10 000 masques automatiques (« faibles ») et leurs photos, pour un pré-entraînement. La base FFHQ complète pèse environ 65 Go : seules les photos utiles ont été téléchargées une par une depuis un miroir Hugging Face (environ 22 Go au total).
- Les rides couvrent en moyenne 0,7 % des pixels d'une image : une classe très rare, ce qui guide le choix de la fonction de perte (section 8).

6.4 Licences

Base	Licence	Conséquence pour le projet
ACNE04	Usage académique libre	Modèle d'acné réservé à un usage académique, citation obligatoire
FFHQ-Wrinkle	CC BY-NC-SA 4.0	Usage non commercial, citation, partage dans les mêmes conditions
FFHQ (photos)	Photos Flickr sous licences Creative Commons	Citation de NVIDIA (Karras et al.)

Les bases ne sont pas republiées dans le dépôt ; le script `training/download_data.py` les retélécharge depuis leurs sources officielles.

7. Modèle d'acné : YOLO11s sur ACNE04

Le modèle d'acné est un détecteur YOLO11s entraîné 10 h 32 min sur ACNE04 ; sur des photos jamais vues, il obtient un mAP@50 de 0,34. C'est en dessous de l'objectif de 0,4 à 0,6 annoncé au départ, et l'analyse en 7.5 explique pourquoi.

7.1 Choix du modèle

- **Détection plutôt que segmentation** : les annotations d'ACNE04 sont des boîtes autour des lésions, et le miroir a besoin du nombre et de la position des boutons.
- **Famille YOLO (Ultralytics YOLO11)** : détection en une seule passe, rapide, poids pré-entraînés sur la base COCO (apprentissage par transfert), export direct au format ONNX pour le Raspberry Pi.
- **Taille « s » plutôt que « m »**, imposée par les 8 Go de la carte graphique en 1 024 px :

Configuration à 1 024 px	Mémoire GPU	Vitesse	Durée estimée pour 150 epochs	Décision
YOLO11m, lots de 2 images	6,8 Go	environ 26 min par epoch	environ 65 h	Écartée, trop long
YOLO11m, lots de 4 et plus	plus de 8 Go			Impossible, mémoire saturée
YOLO11s, lots de 8 images	**environ 5 Go**	**environ 6 min par epoch**	**15 h au maximum**	**Retenue**

Réduire l'image à 640 px pour garder YOLO11m aurait fait disparaître les plus petits boutons : la résolution a été préférée à la taille du réseau. YOLO11s compte 9,4 millions de paramètres.

7.2 Préparation des données (`prepare_acne_data.py`)

1. **Découpage par photo, avant tout découpage en tuiles** : 80 % entraînement, 10 % validation, 10 % test, tirage fixe (graine 42). Deux morceaux d'une même photo ne peuvent donc jamais se retrouver l'un en entraînement et l'autre en test.
2. **Tuiles de 1 024 × 1 024 px à la résolution d'origine**, avec 25 % de chevauchement. Une boîte est gardée dans une tuile si au moins 50 % de sa surface y est visible.
3. **Exemples négatifs** : 30 % des tuiles sans bouton sont conservées, pour apprendre au modèle à ne pas voir de bouton partout.
4. **Photo entière réduite à 1 024 px** ajoutée à chaque photo, pour le contexte du visage.

Partie	Photos	Lésions annotées	Tuiles	Boîtes dans les tuiles
Entraînement	1 166	14 813	12 369	42 042
Validation	146	2 007	1 569	5 723
Test	145	2 163	1 508	5 883

Il y a plus de boîtes que de lésions car une lésion apparaît dans plusieurs tuiles qui se chevauchent et dans la photo réduite.

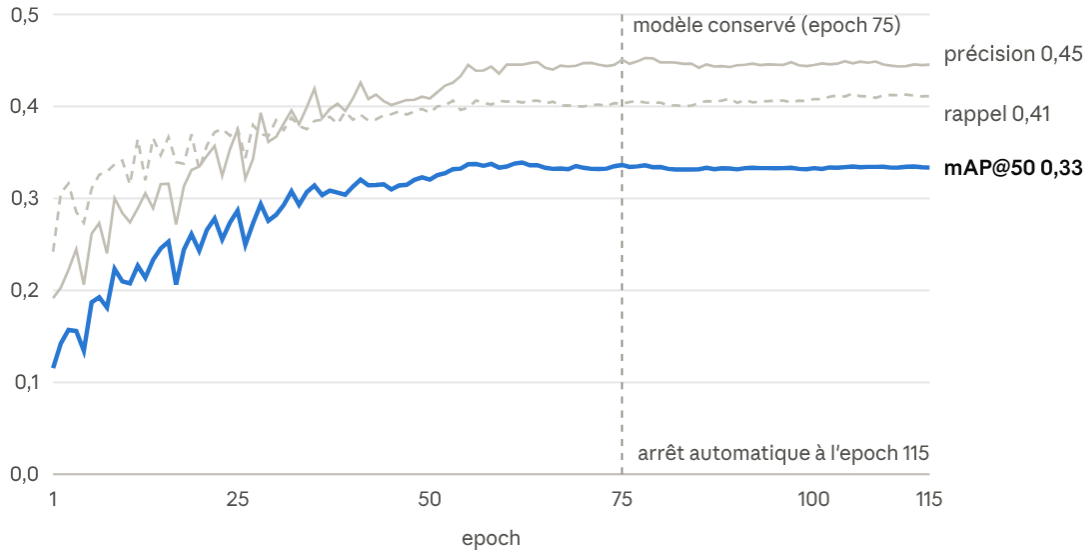
7.3 Entraînement (`train_acne.py`)

Réglage	Valeur	Raison
Poids de départ	YOLO11s pré-entraîné sur COCO	Apprentissage par transfert
Taille d'entrée	1 024 px	Préserver les petites lésions
Epochs / patience	150 au maximum / arrêt après 40 epochs sans progrès	Entraînement long sans surapprentissage inutile
Optimiseur	AdamW, taux d'apprentissage 0,001, décroissance en cosinus, 3 epochs de montée	Convergence stable
Lots	8 images, gradients cumulés jusqu'à 64 (réglage par défaut d'Ultralytics), précision mixte	Mémoire de la RTX 2070
Couleur	teinte ± 0,015, saturation ± 0,5, luminosité ± 0,4	Éclairage et caméra du miroir différents
Géométrie	rotation ± 10°, translation ± 10 %, échelle ± 50 %, miroir horizontal	Pose et distance variables
Mosaïque / mélange	1,0 / 0,1 ; mosaïque coupée sur les 15 dernières epochs	Plus de variété, fin d'entraînement sur des images réalistes

L'entraînement a tourné sous Ubuntu (WSL2) sur la RTX 2070, du 5 octobre 21:16 au 6 octobre 07:49 : 37 921 s, environ 5 lots par seconde, 5 Go de mémoire graphique. Un essai d'une seule epoch avait d'abord validé toute la chaîne (entraînement, évaluation, export ONNX).

Le mAP@50 de validation plafonne vers 0,34 dès l'epoch 55

YOLO11s sur ACNE04, validation : 1 569 tuiles, mesure à chaque epoch



Journal d'entraînement Ultralytics (results.csv) · 115 epochs, 5-6 octobre 2026

Avec cette version d'Ultralytics, le meilleur modèle est choisi sur le mAP@50-95 de validation, qui culmine à 0,087 à l'epoch 75 ; l'arrêt automatique intervient 40 epochs plus tard, à l'epoch 115. Le mAP@50 de validation, lui, culmine à 0,339 à l'epoch 62.

7.4 Résultats sur le test

Le modèle conservé a été évalué une seule fois sur les 145 photos de test (1 508 tuiles), jamais vues pendant l'entraînement.

Métrique	Valeur	Lecture
Précision	0,44	44 % des boutons détectés sont de vrais boutons
Rappel	0,40	40 % des vrais boutons sont trouvés
mAP@50	0,34	Métrique principale de détection
mAP@50-95	0,09	Les boîtes sont peu ajustées à la taille exacte des lésions

Le seuil de confiance de l'application a été fixé à 0,16, la valeur qui maximise le F1 sur la **validation** (F1 = 0,41 à 0,158) et non sur le test. Le modèle est exporté en ONNX (38 Mo, sortie de 5 × 21 504 valeurs par tuile).

7.5 Analyse

- **Le modèle hésite** : ses scores de confiance ne dépassent jamais 0,5, signe d'annotations difficiles (lésions minuscules, très denses sur les acnés sévères).
- **Plus d'entraînement n'aurait pas aidé** : les métriques de validation plafonnent dès l'epoch 55 environ.
- **Le protocole pénalise le score** : le test mélange tuiles en pleine résolution et photos entières réduites, où un bouton ne fait que quelques pixels, alors que l'application n'utilise que des tuiles.
- **Mesure dans l'application** : par photo entière, le nombre de boutons détectés suit le nombre annoté (corrélation 0,75), mais le modèle en trouve environ la moitié (section 9.4).
- **Pistes non lancées** : retirer les photos réduites de l'entraînement et agrandir des tuiles plus petites (version 2, 10 à 15 h d'entraînement estimées).

8. Modèle de rides : U-Net sur FFHQ-Wrinkle

Le modèle de rides est un réseau U-Net entraîné en deux étapes ; sur 100 visages jamais vus, il obtient une IoU de 0,47 et un F1 de 0,64. C'est le plus fiable des deux modèles du projet.

8.1 Choix du modèle

- **Segmentation plutôt que détection** : une ride est une ligne fine et courbe qu'une boîte décrit mal. La segmentation prédit, pour chaque pixel, la probabilité qu'il appartienne à une ride ; on en déduit la forme exacte, la longueur et la répartition par zone.
- **U-Net** : ses connexions entre l'encodeur et le décodeur conservent les détails fins de l'image d'origine, indispensables pour des traits de 3 à 6 pixels de large.
- **Encodeur ResNet-50 pré-entraîné sur ImageNet** (bibliothèque segmentation-models-pytorch) : apprentissage par transfert, utile avec seulement 800 images annotées à la main.

8.2 Découpage des données

Les 1 000 masques manuels sont répartis une fois pour toutes en 800 pour l'entraînement, 100 pour la validation et 100 pour le test (tirage fixe, graine 42, enregistré dans `rides_split.json`). Les 10 000 masques automatiques, sans aucun visage commun avec ces 1 000, servent au pré-entraînement.

8.3 Méthode en deux étapes

La méthode reprend celle de l'article FFHQ-Wrinkle :

1. **Pré-entraînement** sur les 10 000 masques automatiques : 8 epochs, taux d'apprentissage maximal 3×10^{-4}, environ 335 s par epoch.
2. **Ajustement fin** sur les 800 masques manuels : 120 epochs, taux maximal 10^{-4}, environ 27 s par epoch, évaluation sur la validation toutes les 2 epochs et sauvegarde du meilleur modèle (IoU maximale).

Les deux étapes ont tourné le 6 octobre de 07:51 à 09:42 sur la RTX 2070, lancées automatiquement à la fin de l'entraînement de l'acné.

8.4 Réglages

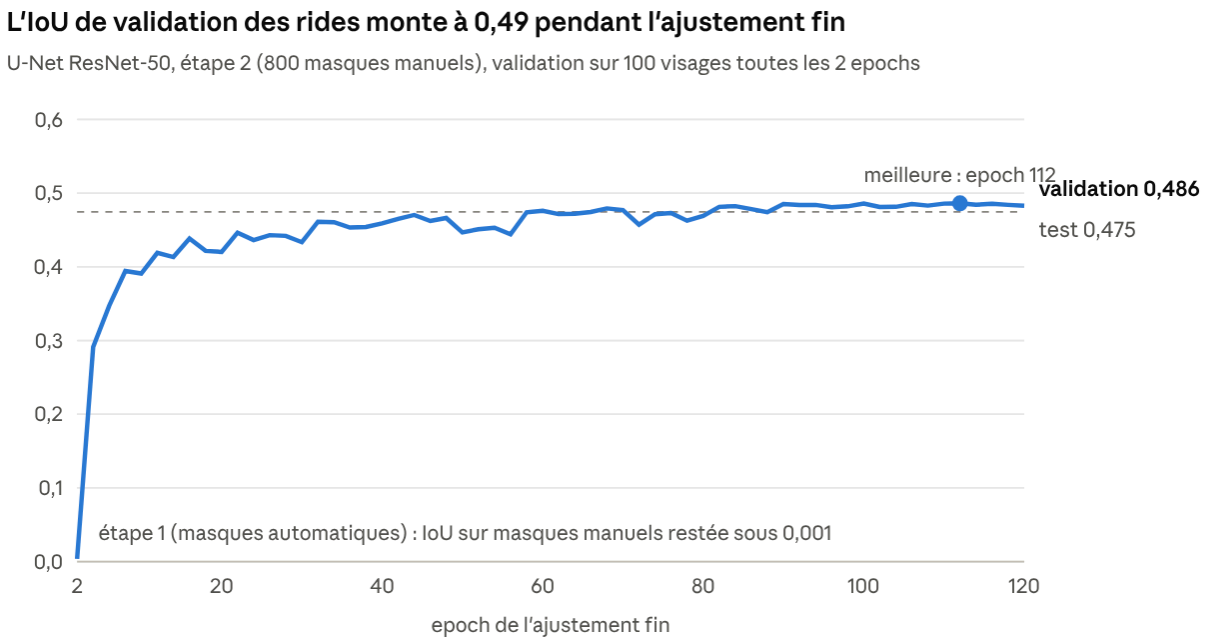
Réglage	Valeur	Raison
Résolution	Recadrages de 512 × 512 px pris dans les images 1 024 × 1 024, sans réduction	Une ride ne fait que 3 à 6 px de large
Choix des recadrages	Une fois sur deux centré sur une zone contenant des rides, sinon au hasard	Les rides ne couvrent que 0,7 % des pixels (moyenne mesurée sur 200 masques)
Augmentation	miroir horizontal ; échelle 0,8 à 1,25, rotation ± 12°, translation ± 5 % ; luminosité et contraste ± 25 % ; teinte, saturation ; bruit, compression JPEG (qualité 50 à 95) ou flou	Robustesse à la caméra et à l'éclairage du miroir
Fonction de perte	Dice + 2 × Focal ($\alpha = 0{,}75$; $\gamma = 2$)	Adaptée à une classe très rare (voir ci-dessous)
Optimisation	AdamW (décroissance 10^{-4}), plan OneCycle (5 % de montée), lots de 8, précision mixte	Convergence rapide et stable sur 8 Go
Évaluation	Images entières 1 024 px, moyenne de l'image et de son miroir	Prédiction plus stable

La perte combine deux termes. Le Dice mesure le chevauchement global entre la prédiction P et la vérité G, sans être dominé par le fond, très majoritaire :

$\text{Dice} = 1 - 2 \times |P \cap G| / (|P| + |G|)$

La Focal donne plus de poids aux pixels difficiles (bords de rides, ridules), p étant la probabilité prédite pour la bonne classe :

Focal = −α × (1 − p)^γ × log(p)



Journal de validation (finetune_log.json) · 60 mesures, 6 octobre 2026 ; IoU de test dans test_metrics.json

Le pré-entraînement n'a rien apporté de mesurable : sur les masques manuels, son IoU est restée sous 0,001, les masques automatiques étant très différents des annotations humaines. C'est l'ajustement fin qui fait le travail : l'IoU passe de 0,29 dès l'epoch 4 à 0,486 à l'epoch 112, avec un plateau après l'epoch 60 environ.

8.5 Résultats sur le test

Le seuil de décision (0,55) a été choisi sur la validation parmi les valeurs de 0,20 à 0,80, puis appliqué tel quel aux 100 visages de test.

Métrique (au pixel, sur tout le test)	Valeur
IoU (indice de Jaccard)	0,475
F1 (Dice)	0,644
Précision	0,628
Rappel	0,661

Pour des traits de quelques pixels, un décalage d'un ou deux pixels compte comme une erreur ; ces scores sont donc bons. Le modèle est exporté en ONNX (130 Mo, entrée 1 × 3 × 1 024 × 1 024). L'export a d'abord échoué (module manquant pour le nouvel exportateur

de PyTorch) ; il a été refait avec l'exportateur classique, sans réentraîner, et l'évaluation a redonné exactement les mêmes scores.

9. Intégration des modèles dans l'application

Les deux modèles tournent dans l'application sans PyTorch, au format ONNX, et chacun reçoit le visage cadré à la même échelle qu'à l'entraînement. Un défaut du masque de peau a été trouvé et corrigé à cette étape, et les scores ont été étalonnés sur des données de test.

9.1 Exécution des modèles

- Les modèles sont lus par **OpenCV DNN** sur le PC et par **onnxruntime** sur le Raspberry Pi (choix fait sur le Pi le 15 octobre), avec OpenCV en secours.
- Vérification sur une même tuile : OpenCV, PyTorch et onnxruntime donnent la même confiance maximale (0,3719), à la quatrième décimale près.
- Si un fichier de modèle est absent, l'application revient automatiquement au détecteur classique correspondant.

9.2 Acné : remise à l'échelle d'ACNE04

Le modèle a appris sur des gros plans où un visage a une taille bien précise. La distance entre les coins externes des yeux a donc été mesurée sur 150 photos ACNE04 tirées au hasard : sur les 140 visages de face, la médiane vaut **860 px** (quartiles 772 et 941 px). À l'utilisation :

1. le visage recadré, à la résolution d'origine de la photo, est remis à l'échelle pour que ses yeux soient écartés de 860 px ;
2. l'image est parcourue en tuiles de 1 024 px qui se chevauchent de 25 %, comme à l'entraînement ;
3. les doublons dans les zones de chevauchement sont fusionnés (suppression des non-maxima, IoU 0,45), avec le seuil de confiance 0,16 ;
4. seules les détections dont le centre tombe sur le visage (hors yeux et lèvres) sont gardées, puis rangées par zone.

9.3 Rides : alignement FFHQ

Le modèle de rides a appris sur des visages cadrés selon l'alignement FFHQ. À l'utilisation, le même cadrage est reproduit à partir du centre des yeux et des coins de la bouche, avec la formule de NVIDIA : le visage est redressé en 1 024 × 1 024 par une transformation

affine, le masque prédit est ramené dans l'image d'origine par la transformation inverse, puis limité au visage. La longueur des rides est mesurée sur leur squelette et rapportée à la surface du visage (mm de rides par cm²).

Contrôle sur 3 visages du test, en passant par l'application : IoU par image de 0,357 à 0,384, contre 0,475 à l'évaluation officielle. L'écart vient du recadrage refait à partir des points du visage, qui décale les traits d'un ou deux pixels. Temps de calcul : environ 3,3 s par visage sur le processeur du PC.

9.4 Étalonnage des scores

Les échelles des scores du modèle profond ont été fixées à partir de mesures faites avec l'application complète :

- **Rides** (100 visages de test FFHQ) : densité médiane 0,905 mm/cm² (25 % : 0,58 ; 75 % : 1,31 ; 90 % : 2,32). Avec une échelle de 2,1, un visage médian obtient 35 (modéré) et les 10 % les plus ridés dépassent 65 (élevé). L'échelle provisoire de 1,0 donnait 59 au visage médian, trop sévère.
- **Acné** (photos de test ACNE04, analysées entières comme sur le miroir) : un visage est détecté sur 125 photos sur 146 (21 profils trop marqués). Le nombre de boutons détectés suit le nombre annoté avec une **corrélation de 0,75**, mais le modèle en trouve environ la moitié : 6,1 en moyenne contre 11,5 annotés (erreur moyenne : 6,0 boutons). L'échelle du score est donc fixée à 3,5 boutons détectés.

Critère	Mesure	Échelle du score	Origine de l'échelle
Acné (YOLO11s)	Boutons détectés	3,5	Étalonnage ACNE04 par photo
Rides (U-Net)	mm de rides par cm² de visage	2,1	Étalonnage FFHQ-Wrinkle
Taches	% de surface couverte	3	Valeur de départ, à calibrer
Pores	Pores par cm²	12	Valeur de départ, à calibrer

9.5 Défaut corrigé : les boutons exclus de leur propre analyse

Au premier essai, aucun bouton n'était trouvé sur une photo qui en montrait clairement. Les coordonnées étaient justes : c'est le masque couleur de peau qui retirait chaque bouton, trop rouge par rapport à la couleur médiane des joues, et creusait un trou à l'endroit exact des lésions. Le test par injection ne l'avait pas vu, car les faux boutons étaient trop pâles.

Correction : les trous entièrement entourés de peau (jusqu'à 40 000 px dans l'image normalisée, soit environ 2 000 mm²) sont rebouchés, tandis que cheveux et fond, qui touchent le bord, restent exclus. Résultat sur la photo de contrôle : 15 boutons détectés au lieu de 0. Le correctif profite aussi aux détecteurs classiques.

10. Partie capteurs du binôme : type de peau

La partie capteurs, réalisée par J. Salek, est intégrée au miroir depuis le 18 octobre : le type de peau qu'elle calcule s'affiche avec les résultats de l'IA, marqué « indicatif ».

10.1 Ce que contient le dépôt d'origine

Le dépôt jihanesalek263-png/SmartSkinAnalyzer contient :

- un programme Arduino Uno qui lit le MLX90614 (température de la peau), le DHT22 (température et humidité de l'air) et un capteur de distance Sharp IR, et envoie une ligne JSON toutes les 2 s ;
- une classification du type de peau par règles fixes ;
- un jeu de 200 lignes généré à partir de ces mêmes règles, et un script de lecture.

Règles d'origine, reprises à l'identique dans `mirror/skin_type.py` :

Condition (dans cet ordre)	Type de peau
Humidité < 30 %	Sèche
Température > 34 °C et humidité > 60 %	Grasse
Température > 34 °C et humidité ⩽ 60 %	Mixte
Température < 31 °C et humidité ⩽ 60 %	Mixte
Autres cas	Normale

Vérification : la reprise donne exactement le même type que l'original sur les 200 lignes de son jeu de données.

10.2 Intégration dans le miroir

- Le miroir reconnaît automatiquement le format de l'Arduino Uno (`distance_cm` , `temp_peau` , `temp_air` , `humidite`) comme celui de l'ESP32 prévu au départ.
- Le type de peau est calculé à la fin de chaque analyse, avec la température mesurée de près (section 12.3), puis affiché sous les mesures des capteurs et enregistré dans l'historique.

- Un conseil adapté au type de peau est ajouté quand la mesure est jugée fiable.

10.3 Limites signalées

- **Ce sont des règles fixées à la main, pas un modèle entraîné.** Le jeu de 200 lignes est généré par ces mêmes règles : les « 100 % de correspondance » ne mesurent aucune précision.
- **Le DHT22 mesure l'humidité de l'air, pas l'hydratation de la peau** : deux personnes dans la même pièce auraient la même « humidité ». Une vraie mesure d'hydratation demanderait un capteur capacitif au contact, comme prévu dans le premier rapport du projet.
- **Le MLX90614 n'est fiable qu'entre 10 et 30 cm** : plus loin, il voit aussi le décor et sous-estime la température. D'où la mesure en deux temps décrite en section 12.3.

11. Matériel et configuration

Le miroir repose sur un Raspberry Pi 4 (caméra, IA, affichage) et un Arduino Uno (capteurs), reliés par un simple câble USB qui alimente aussi l'Arduino. L'Arduino Uno du binôme a été retenu le 18 octobre à la place de l'ESP32 prévu au départ.

11.1 Composants

Composant	Rôle	État au 23 octobre
Raspberry Pi 4 Model B, 4 Go, alimentation 5 V / 3 A	Caméra, analyse IA, affichage, historique	Installé et fonctionnel
Pi Camera Module 3 (capteur IMX708, 12 Mpx, mise au point automatique)	Photo du visage en 4 608 × 2 592 px	Détectée et fonctionnelle
Écran HDMI 5 pouces (800 × 480) derrière un miroir sans tain	Interface du miroir	Interface prête
Arduino Uno R3	Lit les capteurs, pilote la LED, envoie du JSON au Pi	Programme définitif installé
DHT22	Température et humidité de l'air	Fonctionnel (mesures réelles : 27 à 31 °C, 44 à 51 %)
MLX90614 (module GY-906)	Température de la peau, sans contact	Non détecté : fil SDA non relié au module

Composant	Rôle	État au 23 octobre
Sharp GP2Y0A21 (10 à 80 cm)	Distance du visage : déclenchement et guidage	Pas encore monté
LED RGB simple + 3 résistances de 220 Ω	État du miroir (attente, analyse, résultat, erreur)	Pas encore branchée

11.2 Câblage de l'Arduino Uno

Module	Broche du module	Broche de l'Uno	Remarque
MLX90614	VIN / GND	3.3V / GND	Jamais de 5 V sur un capteur nu
MLX90614	SDA / SCL	A4 / A5	Bus I2C, adresse 0x5A ; sur le module, l'ordre est VIN, GND, SCL, SDA
DHT22	Patte 1 / 2 / 4	5V / D2 / GND	Résistance de 10 kΩ entre les pattes 1 et 2 ; patte 3 libre
Sharp GP2Y0A21	Rouge / noir / jaune	5V / GND / A0	Tension analogique convertie en distance
LED RGB	R / G / B	D9 / D10 / D11	Une résistance de 220 Ω par couleur ; patte commune au GND

11.3 Programmes des cartes

Programme	Rôle	Mémoire utilisée
`firmware/uno_capteurs` (retenu)	Repris du programme du binôme, même format JSON ; envoi 4 fois par seconde au lieu de toutes les 2 s, DHT22 lu toutes les 2,5 s ; commandes de LED (`ETAT ATTENTE` , `ETAT ANALYSE` , `LED r g b` , `PING`)	40 % du programme (13 124 octets), 34 % de la RAM (703 octets)

Programme	Rôle	Mémoire utilisée
`firmware/diagnostic_uno`	Programme temporaire : vérifie quels fils sont reliés et alimentés, cherche les modules sur le bus I2C, lit le DHT22	28 % du programme, 22 % de la RAM
`firmware/esp32_capteurs` (variante)	Version ESP32 : détection automatique du VL53L0X ou du VL53L1X, 5 mesures par seconde, LED	26 % du programme, 7 % de la RAM

Les trois programmes compilent sans erreur avec arduino-cli. Les deux programmes Uno ont été téléversés directement depuis le Raspberry Pi, à distance, le 18 octobre.

11.4 Configuration du Raspberry Pi

1. **Système** : Raspberry Pi OS 64 bits écrit avec Raspberry Pi Imager ; nom `skinmirror`, utilisateur `pi`, Wi-Fi, connexion SSH par clé.
2. **Installation** (`deploy/setup_pi.sh`) : paquets système (picamera2, pyserial, evdev, polices), environnement Python avec accès à picamera2, puis OpenCV contrib, scikit-image, Pillow et onnxruntime. MediaPipe n'est pas installé : il plante sur le processeur du Pi 4.
3. **Droits** : l'utilisateur appartient au groupe `dialout` (port série de l'Arduino).
4. **Démarrage automatique** : le miroir se lance en plein écran à l'ouverture de la session (`deploy/start_mirror.sh`) ; son journal est écrit dans `~/skin_history/miroir.log`.
5. **Outils** : arduino-cli installé dans `~/bin` pour reprogrammer l'Uno depuis le Pi ; `tools/lire_arduino.py` affiche ce qu'envoie l'Arduino.

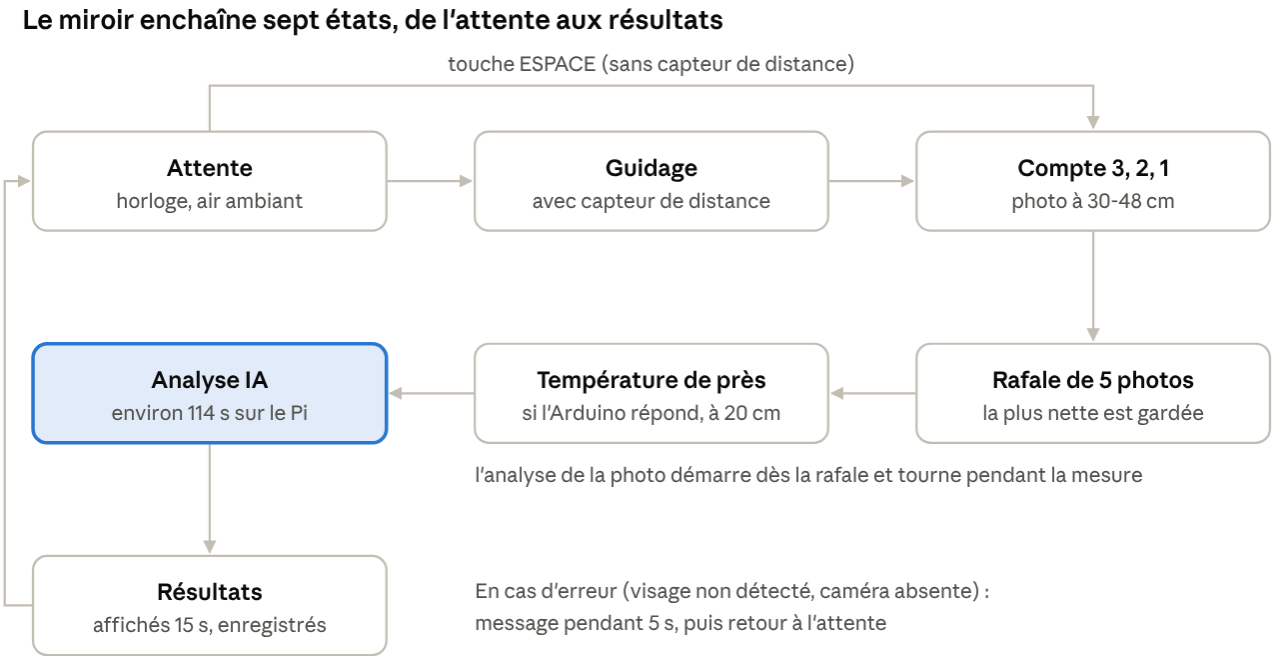
11.5 Règles de montage

- Le MLX90614 et le capteur de distance ne doivent jamais être derrière le verre : le verre bloque l'infrarouge thermique, et le capteur de distance mesurerait le verre.
- Un miroir sans tain ne laisse passer qu'environ 30 % de la lumière : un petit trou dans le film, devant l'objectif, améliore la netteté des photos.
- Le module GY-906 courant voit sous un angle de 90° : loin du visage, il mesure aussi les cheveux et le fond.
- L'interface utilise un fond noir : le noir n'émet pas de lumière, cette zone reste un miroir, et seuls les textes clairs traversent la vitre.

12. Logiciel du miroir

Le programme du miroir (`mirror/`) enchaîne sept états, relie la caméra, l'Arduino et les modèles, et n'affiche que des valeurs réellement mesurées. Il a été testé en simulation sur le PC, puis sur le Raspberry Pi.

12.1 Déroulé d'une analyse



Machine à états du miroir (mirror/app.py) · 7 états

La photo est prise d'abord ; l'analyse IA tourne en arrière-plan pendant la mesure de la température, ce qui n'ajoute aucune attente.

12.2 Fichiers

Fichier	Rôle
`app.py`	Machine à états, déclenchement, assemblage du rapport final
`camera.py`	Pi Camera Module 3 : aperçu en direct, rafale, mise au point sur le visage ; webcam ou « aucune caméra » en secours
`esp32.py`	Lecture du port série dans un fil d'exécution séparé, formats Uno et ESP32, reconnexion toutes les 2 s, commandes de LED
`skin_type.py`	Type de peau par les règles du binôme

Fichier	Rôle
`ui.py`	Écrans sur fond noir (horloge, guidage, compte à rebours, température, analyse, résultats, erreur)
`routine.py`	Conseils simples selon les scores, le type de peau, l'humidité de l'air et la température de la peau
`history.py`	Historique `historique.jsonl` et évolution depuis l'analyse précédente
`keyboard.py`	Clavier lu directement (fenêtre sans focus sur le Pi)
`config.py`	Tous les réglages

12.3 Température de la peau mesurée en deux temps

Le MLX90614 n'est fiable qu'entre 10 et 30 cm, alors que la photo se prend entre 30 et 48 cm pour cadrer tout le visage. Après la photo, le miroir demande donc à la personne de s'approcher à 20 cm et garde la **médiane de 4 lectures** prises toutes les 0,4 s.

- **Avec capteur de distance** : seules les lectures prises entre 10 et 30 cm sont gardées, sous une jauge de distance.
- **Sans capteur de distance** (réglage actuel, `HAS_DISTANCE_SENSOR = False`) : 4 s pour s'approcher, puis seules les lectures plausibles pour une peau proche (28 à 38,5 °C) sont gardées ; une main ou un visage resté loin donne une valeur plus basse et est rejeté.
- Sans lecture valable au bout de 12 s, la dernière valeur est gardée et le type de peau est marqué « peu fiable ».

12.4 Fonctionnement sans capteur de distance

Tant que le Sharp n'est pas monté, l'analyse se lance avec la touche ESPACE, l'étape de guidage est sautée et les résultats restent affichés 15 s. L'entrée A0 de l'Arduino, laissée en l'air, renvoie une distance au hasard (entre 8 et 19 cm relevés) : le miroir l'ignore, pour ne jamais afficher une fausse valeur.

12.5 Caméra

Réglage	Valeur
Aperçu en direct	1 280 × 720 px, mise au point recalée sur le visage toutes les 0,5 s

Réglage	Valeur
Photo	4 608 × 2 592 px (12 Mpx), mise au point figée juste avant
Rafale	5 photos espacées de 0,15 s ; la plus nette sur le visage est gardée (variance du laplacien)
Pose	Plafonnée à 1/40 s pour limiter le flou de bougé, le gain compense (au plus 16)
Image	Retournée horizontalement (effet miroir)

12.6 Robustesse

- **Caméra absente** : le miroir démarre quand même, affiche « Caméra non détectée » et réessaie toutes les 10 s (il plantait au démarrage avant ce correctif).
- **Arduino débranché** : les valeurs des capteurs s'affichent « - » et la reconnexion est automatique.
- **Erreurs pendant l'analyse** : un message s'affiche 5 s, le miroir ne s'arrête jamais.
- **Commandes** : ESPACE lance une analyse ; Échap maintenu 3 s éteint proprement le Pi ; créer le fichier `/tmp/skin_mirror_analyse` lance une analyse à distance.

12.7 Historique et données personnelles

Chaque analyse est ajoutée à `~/skin_history/historique.jsonl` (scores, mesures, type de peau, réglages de la caméra). L'enregistrement des photos est actuellement activé (`SAVE_PHOTOS = True`) : il doit rester réservé aux volontaires qui l'acceptent.

13. Tests et validation

Vingt tests ont été menés, du logiciel seul jusqu'au matériel réel ; il manque encore une analyse complète avec un vrai visage devant la caméra du miroir et une vraie lecture du MLX90614.

13.1 Tests réalisés

Date	Test	Résultat
5 oct.	Injection de faux défauts sur un visage réel	Acné 6/6, taches 3/3, pores 25/25, rides 64 % des pixels

Date	Test	Résultat
5 oct.	Essai YOLO d'une seule epoch (entraînement, évaluation, export)	Chaîne validée ; mAP@50 = 0,09 après 1 epoch, comme attendu
5 oct.	Contrôle des données des rides sur le processeur (découpage, recadrages, sortie du modèle)	Découpage 800/100/100, 10 000 images faibles, dimensions correctes
6 oct.	Évaluation du modèle d'acné sur le test (145 photos)	mAP@50 0,34 ; précision 0,44 ; rappel 0,40
6 oct.	Évaluation du modèle de rides sur le test (100 visages)	IoU 0,475 ; F1 0,644
11 oct.	Même tuile dans OpenCV, PyTorch et onnxruntime	Confiance maximale identique (0,3719)
11 oct.	Rides dans l'application, 3 visages du test	IoU par image 0,357 à 0,384 ; 3,3 s par visage sur le PC
11 oct.	Photo de contrôle ACNE04 dans l'application	0 bouton avant le correctif du masque, 15 après
11 oct.	Acné par photo entière (125 photos de test)	Corrélation 0,75 ; 6,1 détectés contre 11,5 annotés en moyenne
11 oct.	Densité de rides sur 100 visages de test	Médiane 0,905 mm/cm² ; échelle du score fixée à 2,1
11 oct.	Compilation du firmware ESP32	Sans erreur, 26 % de la mémoire
11 oct.	Miroir en simulation sur le PC, scénario automatique avec captures d'écran	Tous les états, rapport enregistré dans l'historique
18 oct.	Lignes au format de l'Arduino Uno et règles du binôme	Format reconnu ; règles identiques sur 200 lignes sur 200
18 oct.	Compilation du firmware Uno	Sans erreur, 40 % de la mémoire
18 oct.	Fusion des modifications faites sur le Pi	330 lignes ajoutées sur le Pi, toutes conservées (2 déplacées)
18 oct.	Analyse complète sur le Raspberry Pi (photo de test, capteurs simulés)	Les deux modèles tournent ; 114 s ; type de peau correct
18 oct.	Filtre de température sans capteur de distance	24 °C (personne loin) : 0 lecture gardée ; 34,2 °C : lectures gardées

Date	Test	Résultat
18 oct.	Photo réelle avec la Pi Camera Module 3	Image 4 608 × 2 592 nette ; caméra encore tournée vers le boîtier
18 oct.	Lecture réelle des capteurs par l'Arduino	DHT22 : 27,4 à 30,9 °C, 44 à 51 % ; MLX90614 : non détecté
18 oct.	Diagnostic du câblage (programme dédié, plusieurs passages)	DHT22 réparé ; SDA du MLX90614 toujours sans liaison après changement de fil

13.2 Temps d'analyse sur le Raspberry Pi 4

L'analyse complète prend 114 s sur le Pi 4, contre environ 5 s sur le PC. Mesure étape par étape, sur la photo de test :

Étape	Temps sur le Pi 4
Chargement des modèles (une seule fois, au démarrage)	10,5 s
Points du visage et zones	1,2 s
Acné (YOLO11s, par tuiles)	43,5 s
Rides (U-Net, image + image miroir)	69,6 s (40,2 s avec un seul passage)
Taches et pores	2,4 s

Le visage de la photo de test ne mesure que 124 px de large ; il est agrandi 14 fois pour l'acné, ce qui multiplie les tuiles. Le temps de l'acné sera à remesurer avec une vraie photo de la caméra. Il a été décidé de garder ce temps pour l'instant ; désactiver le second passage du modèle de rides (`WRINKLE_TTA = False`) ferait gagner environ 30 s.

13.3 Diagnostic du câblage

Le programme de diagnostic force chaque fil à 0 puis vérifie s'il remonte seul : c'est le cas quand un module alimenté y est relié par sa résistance de rappel.

Passage	SDA (A4)	SCL (A5)	MLX90614 sur le bus	DHT22
1	relié	relié	absent	échec
2	relié	non relié	absent	échec

Passage	SDA (A4)	SCL (A5)	MLX90614 sur le bus	DHT22
3	non relié	relié	absent	27,4 °C, 50,6 %
4 (fil SDA changé)	non relié	relié	absent	30,4 °C, 44,9 %

Un seul des deux fils du MLX90614 passe à chaque fois, même après changement du fil SDA. Causes probables, de la plus à la moins probable : broche SDA non soudée sur le module, fil décalé d'une rangée sur la plaque, fil branché sur A3 au lieu de A4.

14. Problèmes rencontrés et solutions

Quinze problèmes ont bloqué le projet à un moment ; tous ont une solution en place sauf le câblage du MLX90614, qui reste à réparer.

Problème	Cause	Solution
MediaPipe refuse de se charger sur le PC	Smart App Control bloque sa bibliothèque non signée	Second moteur de points 100 % OpenCV, choisi automatiquement
PyTorch puis scikit-image bloqués sur le PC	Même cause	Entraînement et exécution sous Ubuntu (WSL2) installé sur D:
Disque C: plein	1,9 Go libres pour environ 30 Go de besoins	Projet, données et environnement déplacés sur D: ; cache pip vidé (1,4 Go)
Base FFHQ complète trop lourde	65 Go pour 70 000 photos	Téléchargement une par une des seules photos utiles (environ 22 Go)
YOLO11m sature la carte graphique	8 Go insuffisants en 1 024 px	YOLO11s, lots de 8 images
Réglage automatique du lot plante	Mémoire saturée pendant ses essais	Taille de lot fixée à la main
Export ONNX des rides échoue	Module manquant pour le nouvel exportateur de PyTorch	Exportateur classique ; scores identiques après export
Aucun bouton détecté dans l'application	Le masque de peau excluait les boutons, trop rouges	Trous entourés de peau rebouchés ; 15 boutons trouvés au lieu de 0

Problème	Cause	Solution
Score de rides trop sévère	Échelle provisoire	Étalonnage sur 100 visages : échelle 2,1
Modifications faites sur le Pi, absentes de GitHub	Travail fait directement sur le Pi depuis un autre appareil	Branche séparée puis fusion ; 330 lignes sur 330 conservées ; sauvegarde de l'ancienne version sur le Pi
Le miroir plantait au démarrage	Caméra non détectée (nappe probablement mal enfoncée)	Message « Caméra non détectée » et nouvel essai toutes les 10 s ; nappe renfoncée, caméra détectée
Fausse distance de 8 à 19 cm affichée	Entrée A0 en l'air, sans capteur	Distance ignorée tant que le capteur n'est pas déclaré
Température lue de trop loin	MLX90614 fiable seulement entre 10 et 30 cm	Mesure en deux temps, avec ou sans capteur de distance
DHT22 muet	Fil DATA non relié à D2 avec sa résistance	Câblage repris : mesures réelles obtenues
MLX90614 non détecté	Fil SDA sans liaison avec le module	**Non résolu** : vérifier la soudure de la broche SDA ou utiliser la broche SDA dédiée de l'Uno

Une erreur a aussi été repérée dans le guide PDF de réalisation pendant la rédaction de ce rapport : il indique que 70 % des recadrages des rides contiennent des rides, alors que le code en choisit un sur deux. Le guide a été corrigé le 23 octobre.

15. Limites, éthique et licences

Le système donne une indication cosmétique et non un diagnostic médical ; ses résultats dépendent de bases publiques dont les photos ne viennent pas de la caméra du miroir.

15.1 Limites techniques

- **Écart de domaine** : ACNE04 et FFHQ ne sont pas des photos prises par la caméra du miroir. La précision réelle doit être vérifiée sur des photos du miroir, sous son éclairage.
- **Acné** : le modèle trouve environ la moitié des boutons ; le niveau de sévérité est fiable, le nombre exact ne l'est pas.

- **Taches et pores** : leurs seuils ont été réglés sans photo annotée et restent à calibrer. Les ombres (ailes du nez, cernes) peuvent être prises pour des taches ; les pores ne sont visibles qu'avec une bonne résolution et une lumière diffuse.
- **Type de peau** : règles fixées à la main sur l'humidité de l'air et la température de la peau ; estimation indicative (section 10.3).
- **Vitesse** : 114 s par analyse sur le Raspberry Pi 4.
- **Biais** : la répartition des carnations, des âges et des types de peau dans les bases influence les performances, à vérifier sur des volontaires variés.

15.2 Éthique et données personnelles

- Le miroir affiche « Indication cosmétique, pas un diagnostic médical » sur chaque écran de résultats, et ses conseils restent généraux.
- Photos de volontaires : consentement écrit, données anonymisées, stockage local sur le Pi. L'enregistrement des photos est activé et doit être réservé aux personnes qui l'acceptent.
- Hygiène : la mesure est entièrement sans contact, contrairement au capteur capacitif envisagé au départ.

15.3 Licences

Élément	Licence ou condition
ACNE04 et le modèle d'acné qui en est issu	Usage académique, citation de Wu et al. (ICCV 2019)
FFHQ-Wrinkle et le modèle de rides	CC BY-NC-SA 4.0 : usage non commercial, citation de Moon et al. (ICPR 2024)
YOLO11 (Ultralytics)	AGPL-3.0
MediaPipe Face Landmarker	Apache 2.0
YuNet, Facemark LBF	OpenCV (Zoo et contrib)

Le dépôt public reprend ces conditions dans son README ; les bases elles-mêmes n'y sont pas publiées.

16. État actuel et prochaines étapes

Au 23 octobre, le logiciel est terminé et installé sur le Raspberry Pi, où le miroir tourne avec la caméra, les deux modèles d'IA et le DHT22 ; il reste surtout du matériel à finir.

Partie	État
Modèles d'acné et de rides	Entraînés, évalués, intégrés, étalonnés
Logiciel du miroir	Terminé, installé et lancé sur le Pi
Caméra	Détectée, photo nette ; à orienter vers le visage
Arduino Uno	Programme définitif installé, relié au Pi
DHT22	Mesures réelles
MLX90614	Non détecté (fil SDA)
Capteur de distance Sharp	Pas encore monté ; le miroir fonctionne sans lui
LED RGB	Pas encore branchée
Code et documentation	Publiés sur GitHub ; guide PDF et ce rapport

Prochaines étapes, dans l'ordre :

- [ ] Réparer la liaison SDA du MLX90614 (soudure de la broche ou broche SDA dédiée de l'Uno), puis vérifier une température de peau réelle (30 à 36 °C main devant).
- [ ] Orienter la caméra vers le visage et faire une première analyse complète avec un vrai visage, en remesurant le temps d'analyse.
- [ ] Monter le capteur Sharp, passer `HAS_DISTANCE_SENSOR` à `True` et vérifier les distances à la règle (20 cm et 40 cm).
- [ ] Brancher la LED RGB sur D9, D10 et D11.
- [ ] Calibrer les seuils des taches et des pores sur des photos prises par la caméra du miroir.
- [ ] Compléter le guide PDF avec le fonctionnement sans capteur de distance et les améliorations faites sur le Pi (rafale photo, clavier, extinction).
- [ ] En option : version 2 du modèle d'acné (sans photos réduites, tuiles agrandies), allègement du modèle de rides pour gagner du temps sur le Pi, publication MQTT pour un tableau de bord.

17. Références

- X. Wu et al., « Joint Acne Image Grading and Counting via Label Distribution Learning », ICCV 2019. Base ACNE04 : github.com/xpwu95/LDL
- J. Moon et al., « Facial Wrinkle Segmentation for Cosmetic Dermatology: Pretraining with Texture Map-Based Weak Supervision », ICPR 2024. Base FFHQ-Wrinkle : github.com/labhai/ffhq-wrinkle-dataset

- T. Karras et al., « A Style-Based Generator Architecture for Generative Adversarial Networks », CVPR 2019 (base FFHQ et procédure d'alignement des visages) : github.com/NVlabs/ffhq-dataset
- O. Ronneberger et al., « U-Net: Convolutional Networks for Biomedical Image Segmentation », MICCAI 2015.
- Ultralytics YOLO11 : docs.ultralytics.com
- Y. Sato et al., « Three-dimensional multi-scale line filter for segmentation and visualization of curvilinear structures in medical images », Medical Image Analysis, 1998 (filtre de crêtes de la phase 1).
- Partie capteurs du binôme : github.com/jihanesalek263-png/SmartSkinAnalyzer
- Code, modèles et documentation du projet : github.com/khaidermohamedd/smart-skin-analyzer